Foundations of AI & ML **Fall 2026**
Professor Amy Greenwald[1] **Supervised Learning Unit**

Perceptrons

The first mathematical model of neural activity was put forth by McCulloch and Pitts in 1943. Six years later, Donald Hebb put forth a model of neural learning that has since been dubbed, "neurons that fire together, wire together:" i.e., if one neuron consistently activates another, then the strength of their connection grows. While some (mostly unsupervised) modern maching learning algorithms are based on Hebbian learning, an alternative mistake-based perspective has come to dominate the supervised learning landscape.

In 1957, Frank Rosenblatt introduced the perceptron learning rule for learning the parameters of a simple perceptron, meaning but one artificial neuron. In this lecture, we describe simple and multilayer perceptrons, the perceptron learning rule, what perceptrons are capable of (classification of linearly separable data), and their fundamental limitations. Despite their simplicity, perceptrons provide fundamental insights into neural computation and serve as building blocks for more complex artificial neural network architectures.

1 Artificial Neural Networks

Motivated by the design of biological brains, artificial neural networks are webs of simple neural processing units interconnected in parallel structures (see Figure 1). The connections among the units at layer l are edges in a directed graph labeled by weights $W^{(l)}$ and biases $\boldsymbol{b}^{(l)}$. Each individual unit is equipped with a simple program that computes a weighted sum of the input data it receives from the units that feed into it, called a **pre-activation**, and outputs the result of applying an **activation function** to the pre-activation.

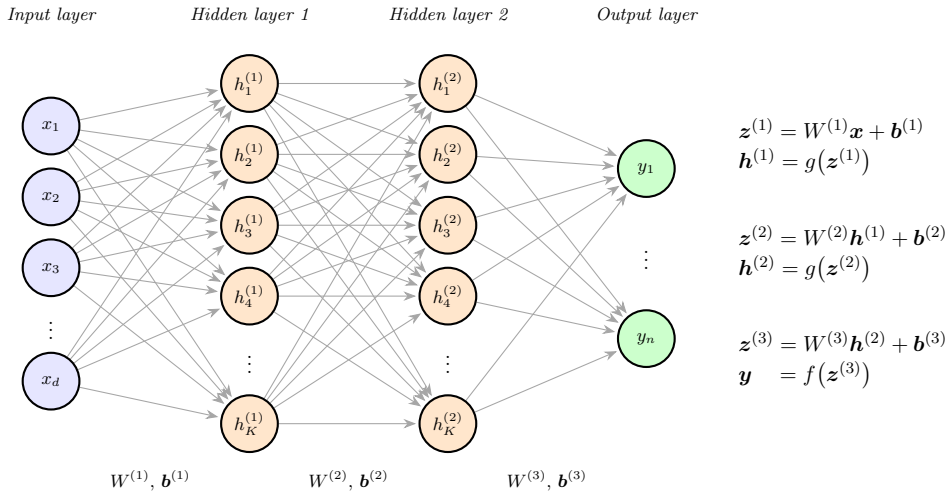


Figure 1: A feed-forward artificial neural network with two hidden layers learning from data set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^m$, with $\boldsymbol{x} \in \mathbb{R}^d$. The output activation f is usually the identity in a regression problem, the sigmoid function in binary classification, and softmax in multiclassification. The activation functions g applied at each hidden layer are often nonlinear, differentiable functions, like tanh or (a smooth version of) ReLU.

[1]This notes were compiled in conjunction with Professor Eric Ewing.

2 Perceptrons

The earliest artificial neurons were **perceptrons**, which are binary threshold units, meaning their activation functions are threshold functions that output one value as long as their input exceeds some threshold, and another value, otherwise. The usual choice of threshold function for the perceptron is the sign function:

$$\text{sign}(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{otherwise} \end{cases} \tag{1}$$

In 1943, McCulloch and Pitts proved that artificial neural networks composed of only binary threshold units are boolean universal machines, i.e., they can compute any boolean function.

The **simple perceptron** has no hidden layers. It is a 1-layer network with only an output layer (Figure 2).

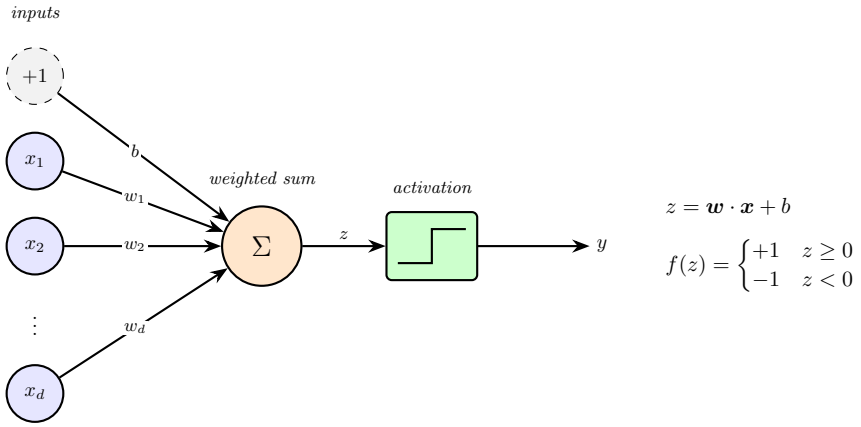


Figure 2: A Simple Perceptron for Binary Classification

Each w_i denotes the strength of the connection from the ith feature of the input to the output. If $w_i > 0$ (resp. $w_i < 0$), the connection is **excitatory** (resp. **inhibitory**), providing a positive influence on the output. The setting $w_i = 0$ implies no connection at all. The parameter b shifts the decision boundary, and is called the bias. The weights and the bias are the parameters an artificial neural network learns.

A simple perceptron can represent a binary classifier, as in Figure 2, or it can be used for multiclassification, as in Figure 3. This latter network computes not just one pre-activation z, but a vector $\boldsymbol{z} \in \mathbb{R}^n$ of pre-activations, where n is the number of classes, or labels. It then outputs the class label with the largest pre-activation. (Alternatively, in topic modeling, where a single input $\boldsymbol{x}$ can be labeled with multiple classes, we can apply a thresholding function to each pre-activation z_i to decide whether to assign $\boldsymbol{x}$ label i or not.)

2.1 The Perceptron Learning Rule

The next question that arises, and one which stymied the research community from 1943 when McCulloch and Pitts first proposed the perceptron model, until 1957, when Rosenblatt came on the scene, is: how do you set the weights and bias of even a simple perceptron? Rosenblatt studied this problem in the context of binary classification, and proposed a simple update called the **perceptron learning rule**.

Rosenblatt's learning rule for the simple perceptron is a **mistake-driven**. If the perceptron's output is correct (i.e., $y = f(z)$), then the weights are left unchanged. But if the perceptron makes a mistake (i.e., $y \neq f(z)$), then the weights are adjusted in the direction of the correct output.

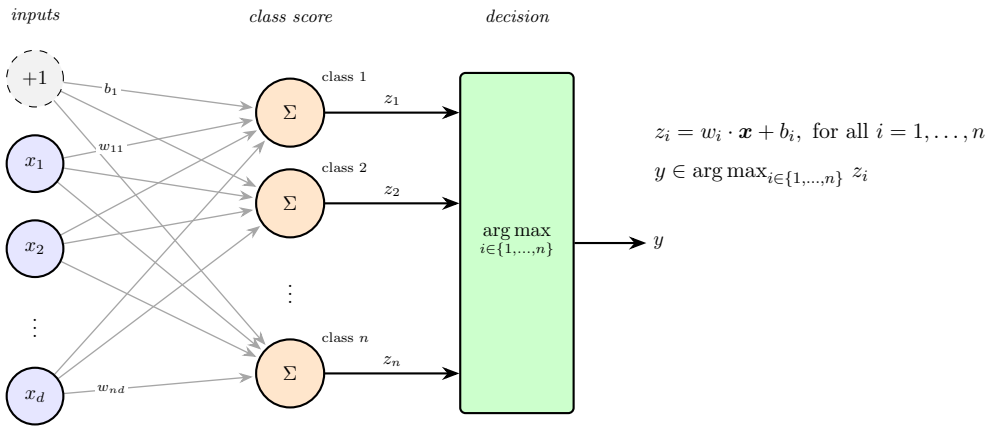


Figure 3: A Simple Perceptron for Multiclassification. In this architecture, the number of hidden units per layer (previously K; see Figure 1) is chosen to be exactly the number of classes n.

Formally, given an observation $\boldsymbol{x} \in \mathbb{R}^d$ with associated label $y \in \{+1, -1\}$, the perceptron rule updates weight w_i, for $i \in \{1, \ldots, d\}$, as follows:

$$\Delta w_i = \alpha \left(y - f(z)\right) x_i \tag{2}$$

where $\alpha \in (0, 1)$ is the learning rate.

Let's confirm that this rule is sensible. The rule is inert when $x_i = 0$. The rule is also inert when the labels match the output. That leaves four cases of interest. First, the input's feature value x_i can be either positive or negative. Second, the label y can be positive while the output is negative; or the label y can be negative, while the output is positive.

The more intuitive case is when x_i is positive, so that its contribution to the weight w_i is supportive. When the label y is positive and the output $f(z)$ is negative, $y - f(z) > 0$, so the update is the product of two positive quantities, which increases w_i, thereby nudging the output towards the positive class. This direction of this nudge is correct, because the label is positive.

On the other hand, when the label is negative and the output is positive, $y - f(z) < 0$, the update is the product of one positive and one negative quantity, which increases w_i, thereby nudging the output towards the negative class. This direction of this nudge is once again correct, because this time the label is negative!

Now consider the case where x_i is negative, and its contribution to the weight w_i is opposing. When the label y is positive and the output $f(z)$ is negative, $y - f(z) > 0$, in which case the update is the product of one negative and one positive quantity, which decreases w_i, thereby nudging the output towards the positive class, *since* $x_i < 0$. This direction of this nudge is correct, since the label is negative.

On the other hand, when the label is negative and the output is positive, $y - f(z) < 0$, in which case the update is the product of two negative quantities, which increases w_i, thereby nudging the output towards the negative class, *since* $x_i < 0$. This direction of this nudge is also correct, since this time the label is negative!

Remark More generally, given a data set $\{(\boldsymbol{x}_i, y_i)\}_{j=1}^m$, with $\boldsymbol{x}_i \in \mathbb{R}^d$ and $y_j \in \{+1, -1\}$, the perceptron rule updates weight w_i, for $i \in \{1, \ldots, d\}$, as follows:

$$\Delta w_i = \frac{1}{m} \sum_{j=1}^{m} \alpha \left(y_j - f(z)\right) x_{ij} \tag{3}$$

where $\alpha \in (0, 1)$ is the learning rate. This rule updates using the average error across all examples.